

中山大学计算机院本科生实验报告

(2024年春季学期)

课程名称：并行程序设计

批改人：

实验	0-环境设置与串行矩阵乘法	专业	计算机科学与技术
学号	22336203	姓名	沈苇杭
Email	shenwh7@mail2.sysu.edu.cn	完成日期	2025.3.26

实验目的

- 掌握如何创建虚拟机，如何使用虚拟机。
- 复习矩阵乘法。
- 掌握矩阵乘法的加速方法。
- 掌握Intel mkl环境配置，学会使用Intel mkl加速程序运行。

实验过程与核心代码

环境配置

按照课件的指引，下载VirtualBox，创建虚拟机，设置CPU数、内存、磁盘空间等，加载虚拟光驱，安装操作系统，并安装OpenMPI。

随后开始安装Intel mkl。首先，使用本机在官网下载Intel mkl的offline installer，随后通过邮箱将它发送到虚拟机中。在虚拟机中，使用命令 `sudo sh` 进行安装，安装完成后使用命令 `source /opt/intel/oneapi/setvars.sh` 启动环境。

此时，尝试运行示例程序，发现虚拟机中没有对应的编译器。于是，在官网下载intel cpp编译器，并通过邮箱发送到虚拟机中，使用命令 `sudo sh` 进行安装。

安装后使用命令 `icx --version` 确认是否安装成功，此时报错 `GLIBC_2.28 not found`。

通过参考博客[五步解决 Ubuntu 18.04 出现GLIBC 2.28 not found的解决方法 没有可用软件包 glibc-2.28。-CSDN博客](#)找到了解决方案。

随后将mkl示例程序通过邮箱发送进虚拟机，保存为.c格式，编译运行，运行成功。

python矩阵乘法

```
import time
import numpy as np

m = 100
n = 150
k = 200

A = np.random.randint(low: 512, high: 2048, size=(m, n))
B = np.random.randint(low: 512, high: 2048, size=(n, k))
C = np.zeros(shape: (m,k), dtype=int)

start_time = time.time()
for i in range(m):
    for j in range(k):
        for k in range(n):
            C[i][j] = C[i][j] + A[i][k] * B[k][j]
end_time = time.time()
time = end_time - start_time
print(f"-f runtime: {time:.2f} seconds")
```

c++矩阵乘法

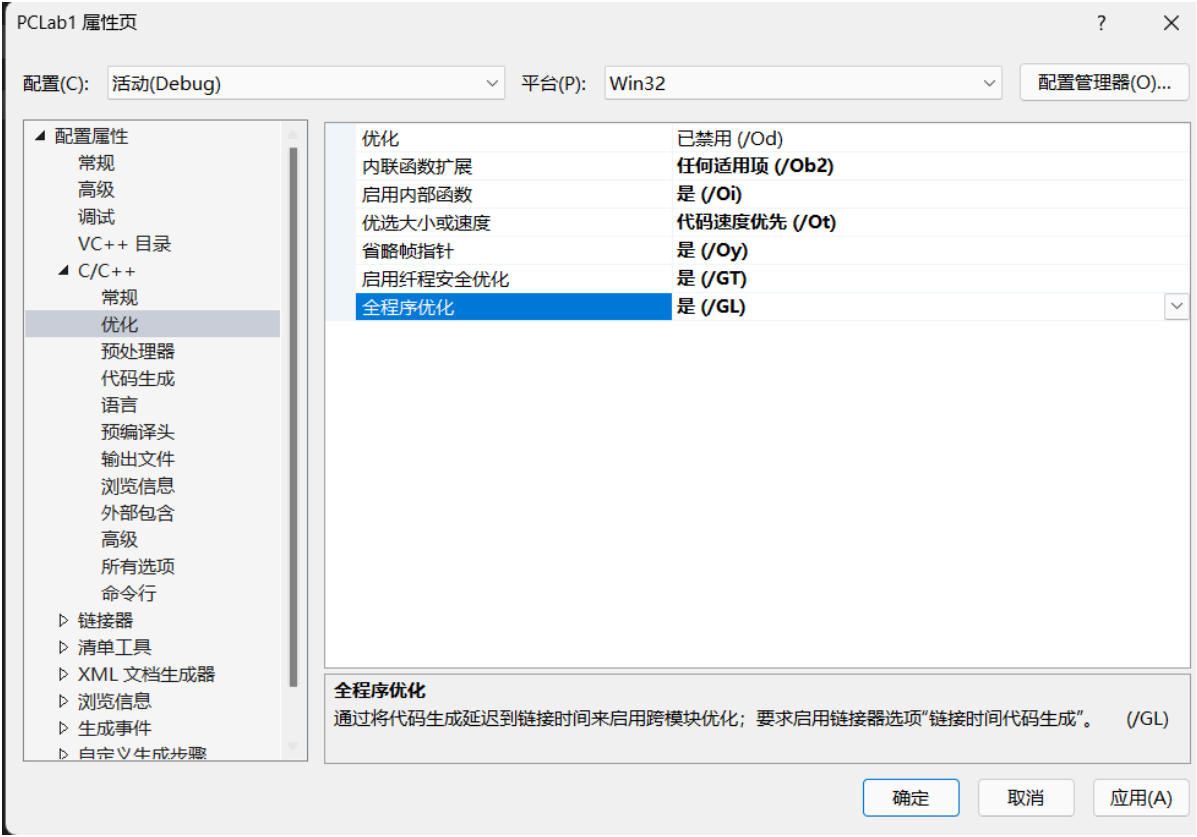
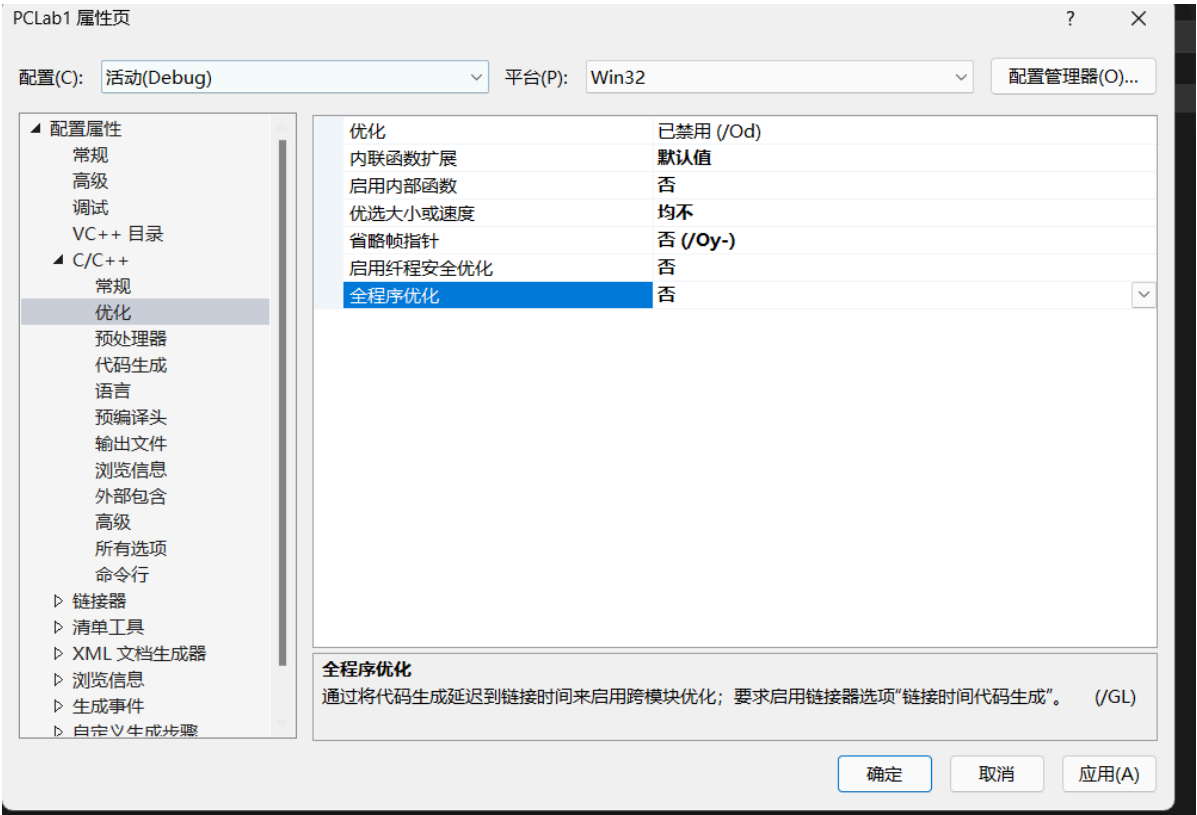
```
int main() {
    int m = 100;
    int n = 150;
    int k = 200;
    vector<vector<int>> A = CreateMatirx(m, n);
    vector<vector<int>> B = CreateMatirx(n, k);
    vector<vector<int>> C = CreateMatirxl(m, k);
    auto start = high_resolution_clock::now();
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < k; j++) {
            for (int a = 0; a < n; a++) {
                C[i][j] = C[i][j] + A[i][a] * B[a][j];
            }
        }
    }
    auto end = high_resolution_clock::now();
    auto time = duration_cast<milliseconds>(end - start);
    cout << "runtime: " << time.count() << "milliseconds" << endl;
    return 0;
}
```

调整循环顺序

```
int main() {
    int m = 100;
    int n = 150;
    int k = 200;
    vector<vector<int>> A = CreateMatirx(m, n);
    vector<vector<int>> B = CreateMatirx(n, k);
    vector<vector<int>> C = CreateMatirxl(m, k);
    auto start = high_resolution_clock::now();
    for (int i = 0; i < m; i++) {
        for (int a = 0; a < n; a++) {
            for (int j = 0; j < k; j++) {
                C[i][j] = C[i][j] + A[i][a] * B[a][j];
            }
        }
    }
    auto end = high_resolution_clock::now();
    auto time = duration_cast<milliseconds>(end - start);
    cout << "runtime: " << time.count() << "milliseconds" << endl;
    return 0;
}
```

在c++中，二维数组通常是按照row-major order存储的，数组的每一行按顺序存储在内存中。也就是说，按行遍历数组比按列更快。

编译优化



编译优化包括：调整编译器选项，设置优化级别，使用 restrict 关键字等。

在vs2022中，可以启用编译器的内联函数扩展、启动内部函数、设置优选速度、省略帧指针、启用线程安全优化和全程序优化。

由于我使用的数据结构是vector, 因此不能使用 restrict 关键字。

循环展开

```
int main() {
    int m = 100;
    int n = 150;
    int k = 200;
    vector<vector<int>> A = CreateMatirx(m, n);
    vector<vector<int>> B = CreateMatirx(n, k);
    vector<vector<int>> C = CreateMatirx1(m, k);
    auto start = high_resolution_clock::now();
    for (int i = 0; i < m; i+=2) {
        for (int a = 0; a < n; a++) {
            for (int j= 0; j < k; j+=2) {
                C[i][j] = C[i][j] + A[i][a] * B[a][j];
                C[i+ 1][j] += A[i+ 1][a] * B[a][j];
                C[i][j+ 1] += A[i][a] * B[a][j+ 1];
                C[i+ 1][j+ 1] += A[i+ 1][a] * B[a][j+ 1];
            }
        }
    }

    auto end = high_resolution_clock::now();
    auto time = duration_cast<milliseconds>(end - start);
    cout << "runtime: " << time.count() << "milliseconds" << endl;
    return 0;
}
```

循环展开是指，将循环体中多次执行的相同操作进行合并，从而减少循环控制的开销。比如，在矩阵乘法中，可以每次循环多计算几个值，这样可以减少循环判断的次数，降低缓存未命中率，从而优化矩阵乘法。

```

/* C source code is found in dgemm_example.c */

#define min(x,y) (((x) < (y)) ? (x) : (y))

#include <stdio.h>
#include <stdlib.h>
#include "mkl.h"
#include <time.h>

int main()
{
    double *A, *B, *C;
    int m, n, k, i, j;
    double alpha, beta;

    m = 100, k = 150, n = 200;
    alpha = 1.0; beta = 0.0;
    clock_t start_time, end_time;

    A = (double *)mkl_malloc( m*k*sizeof( double ), 64 );
    B = (double *)mkl_malloc( k*n*sizeof( double ), 64 );
    C = (double *)mkl_malloc( m*n*sizeof( double ), 64 );

    if (A == NULL || B == NULL || C == NULL) {
        printf( "\n ERROR: Can't allocate memory for matrices. Aborting... \n\n");
        mkl_free(A);
        mkl_free(B);
        mkl_free(C);
        return 1;
    }

    printf (" Intializing matrix data \n\n");
    for (i = 0; i < (m*k); i++) {
        A[i] = (double)(i+512);
    }
}

```

```

for (i = 0; i < (k*n); i++) {
    B[i] = (double)(2048-i);
}

for (i = 0; i < (m*n); i++) {
    C[i] = 0.0;
}

printf (" Computing matrix product using Intel(R) MKL dgemm function via CBLAS interface \n\n");
start_time = clock();
cblas_dgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
            m, n, k, alpha, A, k, B, n, beta, C, n);
printf ("\n Computations completed.\n\n");
end_time = clock();

double time_spent = (double)(end_time - start_time) / CLOCKS_PER_SEC;
printf("Runtime: %f seconds\n", time_spent);

printf ("\n Deallocating memory \n\n");
mkl_free(A);
mkl_free(B);
mkl_free(C);

printf (" Example completed. \n\n");
return 0;
}

```

`cblas_dgemm` 是mkl提供的高效矩阵乘法计算函数，直接通过官网例程即可使用这个函数进行计算。

实验结果

python矩阵乘法

```

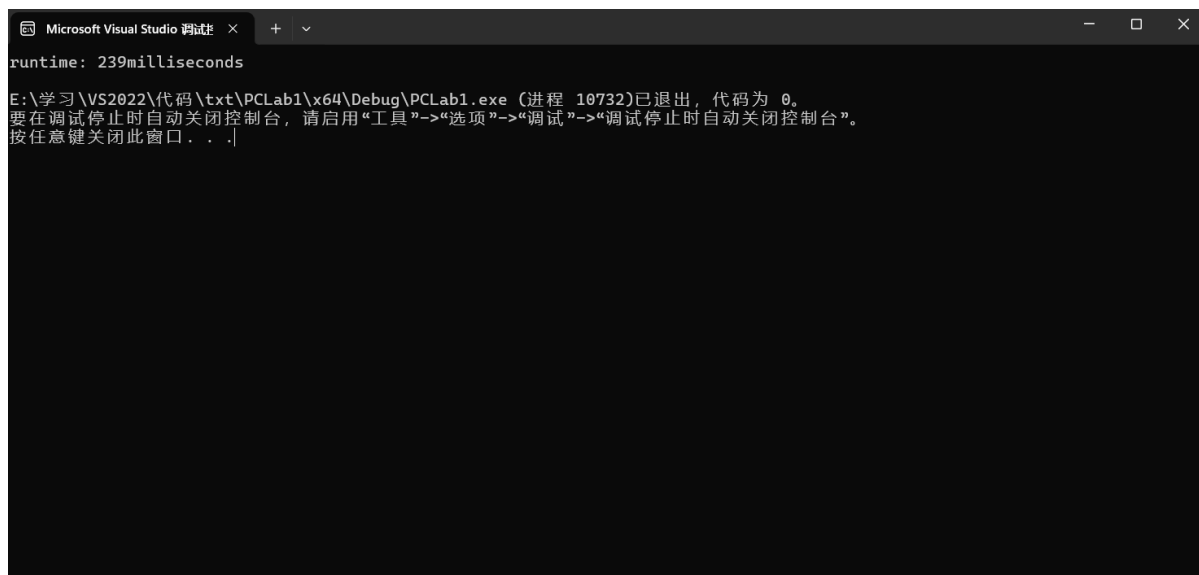
C:\Python312\python.exe "E:\学习\Pycharm\ ParallelComputing\main.py"
-f runtime: 2.35 seconds

Process finished with exit code 0

```

运行时间：2.35s, 2350ms。

c++矩阵乘法



```

Microsoft Visual Studio 调试
runtime: 239milliseconds
E:\学习\VS2022\代码\txt\PCLab1\x64\Debug\PCLab1.exe (进程 10732)已退出，代码为 0。
要在调试停止时自动关闭控制台，请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
按任意键关闭此窗口。 . . .

```

运行时间：239ms。

调整循环顺序

```
runtime: 156milliseconds
```

E:\学习\VS2022\代码\txt\PCLab1\x64\Debug\PCLab1.exe (进程 10276)已退出，代码为 0。
要在调试停止时自动关闭控制台，请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
按任意键关闭此窗口...|

运行时间：156ms。

编译优化

```
Microsoft Visual Studio 调试  X + v  
runtime: 148milliseconds
```

E:\学习\VS2022\代码\txt\PCLab1\x64\Debug\PCLab1.exe (进程 2520)已退出，代码为 0。
要在调试停止时自动关闭控制台，请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
按任意键关闭此窗口...|

运行时间：148ms。

循环展开

```
Microsoft Visual Studio 调试  X + v  
runtime: 84milliseconds
```

E:\学习\VS2022\代码\txt\PCLab1\x64\Debug\PCLab1.exe (进程 16260)已退出，代码为 0。
要在调试停止时自动关闭控制台，请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
按任意键关闭此窗口...|

运行时间：84ms。

Intel MKL

```
paimon@PCLab1:~$ icx -qmk=parallel '/home/paimon/下载/LAB1-2.c' -o lab1-2_mkl  
paimon@PCLab1:~$ ./lab1-2_mkl  
Intializing matrix data  
  
Computing matrix product using Intel(R) MKL dgemm function  
via CBLAS interface  
  
Computations completed.  
  
Runtime: 0.027202 seconds  
  
Deallocating memory  
  
Example completed.  
  
paimon@PCLab1:~$
```

运行时间：27ms。

实验结果统计表

版本	实现描述	运行时间	相对加速比	绝对加速比	浮点性能
1	Python	2.350s	/	/	0.0026
2	C/C++	0.239s	883%	883%	0.0251
3	调整循环顺序	0.156s	53%	1406%	0.0385
4	编译优化	0.148s	5%	1487%	0.0405
5	循环展开	0.084s	76%	2698%	0.0714
6	Intel MKL	0.027s	211%	8604%	0.2222

实验感想

在本次实验中，我遇到的主要问题是虚拟机使用和mkl环境配置的问题。在第一次创建虚拟机时，我设置的磁盘空间太小了，导致安装mkl时报错磁盘空间不足。查阅扩容方法、尝试在本机windows中配置vs2022+mkl环境后，发现重新创建一个虚拟机是最简单的方法。

安装好mkl，尝试运行程序时，我又遇到了编译器缺失的问题。我的虚拟机里只有gcc，没有intel的cpp编译器。安装好编译器后，又遇到了 GLIBC_2.28 not found 的问题。解决这些问题后，实际编程的过程比较顺利。

在本次实验中，我学到了如何创建虚拟机，以及如何在虚拟机中运行代码，同时掌握了矩阵乘法的几种优化方法。